



UNIVERSITÀ DEGLI STUDI
DI SALERNO



Elementi Di Ingegneria Dei Linguaggi Di Programmazione:



SCARTELLATO

// 'o compilatore ca porta ciorta

Candidati:

Stefano Santoro 0512120778

Renato Natale 0512119641

Unisa, 2026

Contents

1	Introduzione al linguaggio	1
1.1	Motivation	1
1.1.1	Architettura e Pipeline del Compilatore.	2
1.1.2	Hello World!	2
2	Specifiche del linguaggio	4
2.1	Anatomia di un programma	4
2.2	Notazione base	5
2.2.1	Istruzioni di blocco e di scope	5
2.2.2	Istruzioni matematiche	5
2.2.3	Assegnamento	6
2.3	Scelta dei Tipi Primitivi e del Sistema di Tipizzazione	6
2.3.1	Tabella dei tipi	7
2.3.2	Concatenazione tra i tipi	7
2.3.3	Swap	9
2.3.4	ArrayType	9
2.3.5	Applicazione del tipo generico Burdell	11
2.3.6	Semplici costrutti primitivi OOP.	13
2.4	Gestione dello Scope	14
3	Esempi di applicazione del linguaggio con relativa traduzione in C	16
3.0.1	Fibonacci iterativo.	17
3.0.2	Fibonacci ricorsivo	18
3.0.3	Bubble Sort.	20
4	Utilizzo dell'IA come Strumento di Supporto nello Sviluppo	22
4.1	Filosofia d'Uso: L'IA come Tool e non come Sostituto dell'Uomo	22
4.2	Supporto nell'Implementazione del Pattern Visitor	22
4.3	Il Transpiler: Traduzione dal Codice Scartellato al Linguaggio C e Gestione dei Compiti Ripetitivi.	23
4.4	Supporto nel Debugging e Correzione della Traduzione dei Tipi Generici .	23
4.5	Automazione della Compilazione tramite GCC	23

4.6	Apprendimento e Supporto nel Testing del Compilatore.	24
5	Testing e Risultati finali	25

1

Introduzione al linguaggio

1.1 Motivation

Nel corso degli ultimi anni, lo studio e lo sviluppo dei linguaggi di programmazione hanno continuato a rappresentare un pilastro fondamentale dell'informatica. La progettazione di un nuovo linguaggio richiede uno sforzo noto: da un lato, la rigorosità formale della sintassi e della semantica per garantire una corretta esecuzione; dall'altro, la necessità di fornire astrazioni logiche strutturate. La creazione di un compilatore funzionante richiede infatti un'attenta implementazione e orchestrazione di fasi critiche, dall'analisi lessicale e sintattica fino al rigido controllo semantico e alla generazione del codice.

In questo contesto, la realizzazione di linguaggi sperimentali, dialettali o "esoterici" assume un'importante valenza didattica e ingegneristica. Progettare un linguaggio con una sintassi non convenzionale permette di esplorare a fondo i meccanismi interni dei compilatori, mettendo alla prova la flessibilità degli strumenti di parsing, la gestione degli scope e la robustezza del sistema dei tipi (statico o dinamico). Questo approccio dimostra empiricamente che le fondamenta teoriche dell'ingegneria dei linguaggi rimangono invariate e solide a prescindere dal vocabolario utilizzato.

Da queste premesse nasce *Scartellato*, un linguaggio di programmazione dotato di un proprio compilatore, caratterizzato da una forte e marcata identità partenopea. L'adozione di un registro lessicale ironico e profondamente ancorato alla tradizione napoletana non deve tuttavia trarre in inganno per la sua apparente leggerezza linguistica. Al contrario, il progetto ha una solida base tecnica e vuole dimostrare come la complessa progettazione di un vero compilatore possa essere applicata con successo a un linguaggio "folkloristico". L'obiettivo finale è provare che anche una terminologia estremamente semplice e a tratti "simpatica" possa essere accostata e usata a costrutti complessi e astratti della programmazione, rendendo la programmazione un'attività più leggera e meno rigorosa da praticare.

1.1.1 Architettura e Pipeline del Compilatore

Per trasformare un codice sorgente scritto in *Scartellato* in un modulo eseguibile o tradotto, la struttura del compilatore è stata progettata seguendo una pipeline classica suddivisa in quattro fasi sequenziali e modulari:

1. **Analisi Lessicale (Lexer):** Il sorgente viene analizzato come un flusso di caratteri e convertito in una sequenza di *token*. In questa prima fase vengono isolate le stringhe, i valori numerici, i delimitatori di istruzione (come il punto esclamativo !) e riconosciute le parole chiave distintive del linguaggio (es. *Uè, Mestier, Robba, Burdell*).
2. **Analisi Sintattica (Parser):** A partire dal flusso di token, il parser verifica la conformità del codice rispetto alla grammatica formale definita per *Scartellato*. Il prodotto finale di questa fase è la costruzione dell'Albero della Sintassi Astratta (*Abstract Syntax Tree* – AST), che rappresenta in modo gerarchico le relazioni tra moduli, funzioni e dichiarazioni.
3. **Analisi Semantica e Tabella dei Simboli:** L'AST viene attraversato per validare la coerenza logica del programma. L'analizzatore semantico popola la *Symbol Table*, gestisce la visibilità delle variabili (*scope*) e controlla la compatibilità dei tipi, garantendo la coesistenza flessibile tra i tipi espliciti (es. *numr*) e il tipo generico *Burdell*.
4. **Generazione del Codice e Gestione del Runtime:** Nell'ultima fase, l'AST validato viene tradotto nel linguaggio target C grazie a un **Transpiler**. In questo livello vengono gestite a basso livello le strutture dati in memoria (come l'accesso agli array tramite notazione posizionale).

1.1.2 Hello World!

Questo report ha l'obiettivo di informare l'utente dell'architettura del sistema e renderlo in grado dopo la lettura di scrivere un programma funzionante in *Scartellato* guidandolo anche avendo zero dimestichezza in materia.

Partiamo con il classico esempio della programmazione : «*Hello World!*», o meglio dire in napoletano «*Ciao waju!*».

Esempi di codice

```
vacant Uè)( }  
    allucc )??Ciao Waju??( !  
}
```

questo corrisponde al seguente codice in C

```
int main( ) }  
    printf("Hello World");  
}
```

Analizziamo il codice in questione

- vacant Uè : corrisponde alla funzione **main**, la funzione che deve essere usata per mandare in compilazione il programma
- allucc: corrisponde alla funzione di **stampa** in standard output, come in molti linguaggi essa accetta una stringa ed è possibile usarla per stampare il contenuto delle variabili

2

Specifiche del linguaggio

2.1 Anatomia di un programma

Nonostante *Scartellato* sia un linguaggio semplice da comprendere non ne pregiudica l'efficacia e l'utilità applicativa. Il linguaggio si distingue per un'elevata versatilità: grazie a un equilibrato livello di astrazione, permette di sviluppare agevolmente programmi complessi evitando una sintassi verbosa come ad esempio nei linguaggi C, C++, java . Vi è infatti, la possibilità spaziare dai costrutti logici fondamentali fino ai paradigmi della programmazione orientata agli oggetti (OOP), come classi e oggetti, trattati nativamente in maniera essenziale per non appesantire eccessivamente l'architettura del linguaggio. Lo spettro espressivo del linguaggio spazia dai costrutti logici fondamentali fino ai paradigmi della programmazione orientata agli oggetti (OOP). Il supporto a classi e oggetti è integrato nativamente in una declinazione snella, progettata per coprire i pattern di utilizzo principali senza appesantire l'architettura del linguaggio.

Di seguito viene illustrata una panoramica dei più rilevanti elementi sintattici del linguaggio, ciascuno dei quali sarà successivamente approfondito nel corso del documento analizzando la rispettiva struttura formale e regole di utilizzo:

Costrutto	Descrizione
Uè	Rappresenta il <i>main</i> , ovvero il punto di ingresso fondamentale per l'esecuzione e la compilazione del programma.
Mestier	Rappresenta la keyword per dichiarare una funzione.
jamm ja :	Rappresenta la keyword per effettuare una chiamata a funzione.
Robba	Rappresenta la keyword per dichiarare una classe.
o_mast	Rappresenta la keyword per dichiarare un costruttore, che l'oggetto dovrà richiamare nella funzione main Uè.
Type] [	Rappresenta la notazione per dichiarare la struttura dati array.
Burdell	Rappresenta la keyword per dichiarare un tipo generico.

Tabella 2.1: Sintassi logica e parole chiave principali del linguaggio *Scartellato*.

2.2 Notazione base

2.2.1 Istruzioni di blocco e di scope

In *Scartellato* le istruzioni sono racchiuse come in altri famosi linguaggi di programmazione dalle istruzioni parentesi . Per conferire un'identità sintattica peculiare al linguaggio, si è scelto di invertirne l'orientamento convenzionale:

- **le istruzioni racchiuse in }** { parentesi graffe servono per indicare che si entra in un blocco soggetto a scope
- **le istruzioni racchiuse in) (** parentesi tonde servono per indicare una lista di parametri se si è in una funzione

2.2.2 Istruzioni matematiche

Per le istruzioni matematiche viene adottata anche qui la stessa scelta

- **il token -** sta a indicare il simbolo matematico per l'addizione
- **il token +** sta a indicare il simbolo matematico per la sottrazione
- **il token *** sta a indicare il simbolo matematico per la divisione
- **il token /** sta a indicare il simbolo matematico per la moltiplicazione
- **il token %** non è invertito rappresenta il classico modulo matematico

- il token `++` non è invertito e rappresenta l'operatore incremento
- il token `--` non è invertito e rappresenta l'operatore di decremento

2.2.3 Assegnamento

Per le istruzioni di assegnamento vengono adottate queste scelte

- il token `=` sta a indicare la classica assegnazione delle variabili
- il token `+=` sta a indicare il simbolo di assegnazione e somma
- il token `-=` sta a indicare il simbolo di assegnazione e sottrazione
- il token `*=` sta a indicare il simbolo di assegnazione e divisione
- il token `/=` sta a indicare il simbolo di assegnazione e moltiplicazione

2.3 Scelta dei Tipi Primitivi e del Sistema di Tipizzazione

Nel panorama dei linguaggi di programmazione e dei compilatori, la scelta dei tipi primitivi supportati nativamente dal linguaggio rappresenta spesso un bivio netto. Da un lato vi sono linguaggi fortemente e staticamente tipati come il **C**, caratterizzati da una rigida e preventiva definizione del tipo di dato; dall'altro vi sono linguaggi come **Python** o **JavaScript**, in cui il tipo è dinamico e dipende completamente dal valore assegnato alla variabile in quel momento.

In questo contesto, **Scartellato** adotta un approccio fortemente versatile e rivoluzionario, rifiutando una separazione netta. Il linguaggio non impone una scelta esclusiva, ma offre una soluzione ibrida: permette sia di dichiarare variabili in modo generico e fluido (in stile Python), sia di specificare tipi di dato espliciti e vincolanti (in stile C).

La logica alla base di questa architettura risiede nella volontà di offrire all'utente la massima flessibilità possibile, senza costringerlo a rispettare regole sintattiche eccessivamente stringenti fin da subito.

2.3.1 Tabella dei tipi

Costrutto	Descrizione
numr	Tipo di dato primitivo per la memorizzazione di un numero generico.
nbruogglio	Tipo di dato per la gestione delle stringhe di testo.
lettr	Tipo di dato per la memorizzazione di un singolo carattere.
burdell	Tipo dinamico universale: può accogliere e gestire in esecuzione qualsiasi tipo primitivo presente in questa tabella.
lota	Keyword per la definizione del valore booleano. Può assumere sasicchj (TRUE) friariell (FALSE)
Type][	struttura dati array per il tipo specificato.

Tabella 2.2: Tipi primitivi del linguaggio *Scartellato*.

2.3.2 Concatenazione tra i tipi

La concatenazione nel linguaggio rispetta regole particolari in base ai tipi con cui viene effettuata. Si usa l'operatore "+" **che sta a indicare la concatenazione**. Ecco un elenco delle operazioni ammesse:

- `nbruogglio + nbruogglio = nbruogglio`
- `nbruogglio + numr = nbruogglio`
- `numr + nbruogglio = nbruogglio`
- `nbruogglio + lettr = nbruogglio`
- `lettr + nbruogglio = nbruogglio`

Ulteriori concatenazioni diverse rispetto a quelle appena elencate portano il programma in uno stato erraneo causando una BOTT A MUR Exception

Esempi di codice

1) caso

```
nbruogglio s = ??pasta ?? !  
nbruogglio d = ??e patane??!  
s= s+d !  
OUTPUT: pasta e patane
```

2) caso

```
nbruogglio s = ??ciro ?? !  
numr d = 98!  
s= s+d !  
OUTPUT: ciro 98
```

3) caso

```
numr x = 98 !  
nbruogglio d = ?? ciro?? !  
d= x + d !  
OUTPUT: 98 ciro
```

4) caso

```
nbruogglio x = ??pizza ?? !  
lettr d = ?p? !  
x= x+d !  
OUTPUT: pizza p
```

5) caso

```
lettr x = ?p? !  
nbruogglio d = ??pizza?? !  
d= x+d !  
OUTPUT: ppizza
```

2.3.3 Swap

il linguaggio permette nativamente tramite il TOKEN di swap (<->) di scambiare i valori di due variabili a patto che siano dello stesso tipo.

Esempi di codice

```
numr s = 9!  
numr k = 24!  
k <-> s !  
OUTPUT: k= 9, s=24
```

2.3.4 ArrayType

Gli array in Scartellato non sono semplici contenitori di dati come in C , ma strumenti dinamici e potenti. Il loro punto di forza è la capacità di gestire operazioni logiche astratte in modo estremamente corto e pulito. Grazie a questa gestione, è possibile condensare logiche di manipolazione dati all'interno di **un'unica riga di codice**.

Per creare un array in Scartellato bisogna usare la seguente notazione `Type ] [` dove `Type` rappresenta uno dei tipi visti in precedenza:

Le operazioni native di un array sono

1. assegnazione + append
2. rimozione di un elemento già inserito
3. assegnazione in una posizione specifica
4. visita di una posizione specifica

Vediamoli nel dettaglio

1) Assegnazione di un elemento

Per assegnare un elemento in un array si bisogna usare il **token -=**. Questa sintassi permette nativamente di assegnare un valore alla fine dell'array appena creato.

Esempi di codice

<code>numr eta = 22 !</code>	assegnazione valore alla variabile di tipo numr
<code>numr] [s !</code>	creazione dell'array
<code>s -= eta !</code>	assegnazione in append
<code>s -= 8 ! ...</code>	

Dopo questa assegnazione in memoria ci sarà una rappresentazione simile:

s	eta	numero
[array]	22	8

Tabella 2.3: Simulazione dell'allocazione dei dati nelle celle di memoria.

Per assegnare un elemento in un array in una posizione specifica bisogna usare l'indice posizionale, caratterizzato da un valore numerico racchiuso nella sintassi dedicata, come ad esempio] 1 [. Gli indici partono da 0 fino al valore N-1.

Esempi di codice

numr eta = 22 !	assegnazione valore alla variabile di tipo numr
numr][s !	creazione dell'array
s -= eta !	assegnazione in append
s]0[= 8 !	assegnazione usando l'indice posizionale
...	

Dopo questa assegnazione in memoria ci sarà una rappresentazione simile:

s	numero
[array]	8

Tabella 2.4: Simulazione dell'allocazione dei dati nelle celle di memoria.

2) Rimozione di un elemento

Per assegnare un elemento in un array bisogna usare il **token +=**. Questa sintassi permette nativamente di rimuovere un valore solamente se presente.

Esempi di codice

numr eta = 22 !	assegnazione valore alla variabile di tipo numr
numr][s !	creazione dell'array
s -= eta !	assegnazione in append
s += eta !	rimozione dell'elemento
...	

Dopo questa operazione l'array rimarrà allo stato iniziale vuoto.

Attenzione!

Usare in maniera errorea questa sintassi come assegnando alla variabile un valore non presente nell'array genererà una `BOTT A MUR EXCEPTION` che indica una azione fatale che porterà alla terminazione del programma.

3) uso dell'indice posizionale

Negli array in *Scartellato*

Esempi di codice

<code>numr eta = 22 !</code>	assegnazione valore alla variabile di tipo <code>numr</code>
<code>numr][s !</code>	creazione dell'array
<code>s -= eta !</code>	assegnazione in append
<code>s += eta !</code>	rimozione dell'elemento
<code>...</code>	

Dopo questa operazione l'array rimarrà allo stato iniziale vuoto.

Attenzione!

Usare in maniera errorea questa sintassi come assegnando alla variabile un valore non presente nell'array genererà una `BOTT A MUR EXCEPTION` che indica una azione fatale che porterà alla terminazione del programma.

2.3.5 Applicazione del tipo generico `Burdell`

Una delle caratteristiche più potenti e flessibili di *Scartellato* è senza dubbio il tipo generico `burdell`. A differenza dei tipi di dato statici tradizionali, che vincolano una variabile a un'unica natura per tutto il suo ciclo di vita, `burdell` agisce come un **contenitore dinamico universale**.

Questa particolarità permette a una singola variabile di accogliere, mutare e gestire in fase di esecuzione qualsiasi tipo primitivo (come `numr` o `nbruoglio`) senza generare errori semantici. Si rivela quindi lo strumento ideale per gestire dati eterogenei, scrivere algoritmi più astratti o affrontare situazioni in cui il tipo di dato in ingresso non è rigidamente noto a priori.

Per comprendere appieno la potenza e il comportamento di questa struttura, nei paragrafi successivi analizzeremo nel dettaglio la sua sintassi attraverso una serie di **esempi pratici ed esplicativi**, illustrando i principali scenari di utilizzo e le migliori pratiche per applicare `burdell` all'interno del proprio codice.

Esempi di codice

```

numr x = 100 !
lota bool = sasicchj!
nbruogglio stringa = ??ciao??!
burdell variabiledinamica = x !
variabiledinamica = stringa!
variabiledinamica = bool !
OUTPUT : sasicchj
...

```

Inizia come numr
diventa nbruogglio
diventa lota

Burdell & ArrayType

è possibile fare interagire la variabile generica come se fosse un elemento di un array se corrisponde al tipo dell'ArrayType

Esempi di codice

```

...
numr x = 100 !
nbruogglio ][ arr !
arr -= ??ciao?? !
lota bool = sasicchj!
burdell variabiledinamica = x !
variabile_dinamica = arr]0[!
variabile_dinamica= ??mondo?? !
arr -= variabile_dinamica!
OUTPUT : valore variabile dinamica: mondo.
Valore array]1[ = mondo

```

Inizia come numr
assegnazione elemento array

Esempi di codice

```
...
numr x = 100 !
numr ][ arr !
arr -=2 !                               indice 0 assegnazione val 2
burdell variabile_dinamica = ??sono stringa??!
variabile_dinamica = arr]0[!           assegnazione elemento array
variabile_dinamica- - !
OUTPUT : valore variabile dinamica: 1
```

- **Assunzione di comportamento:** Una volta che un burdell ha assunto un tipo concreto (es. gli hai assegnato un numero), può subire tutte le operazioni consentite per quel tipo (somme, sottrazioni, concatenazioni, ecc.).
- **Incremento/Decremento:** accetta esplicitamente gli operatori unari ++ e -- se assume un valore di tipo "numr".
- **Operazioni miste (Stringhe e Numeri):** Se un burdell assume il tipo "numr", e lo si somma a un "nbruogglio" con gli operatori += o -=, il burdell muterà il suo tipo interno in "nbruogglio" in modo automatico.

2.3.6 Semplici costrutti primitivi OOP

Il linguaggio non rinuncia a fornire astrazioni di alto livello, implementando un supporto basilare per il paradigma della Programmazione Orientata agli Oggetti (OOP). L'introduzione di questi costrutti permette allo sviluppatore di abbandonare l'approccio puramente procedurale per organizzare il codice memorizzando comportamenti specifici per gli oggetti. Il linguaggio quindi supporta:

- **Dichiarazione della classe (Robba):** la definizione formale del "modello" architettonico (il *blueprint*), all'interno del quale vengono raggruppate le variabili di istanza (stato) e le funzioni (comportamento).
- **Istanziamento degli oggetti:** il processo di allocazione e creazione di un oggetto concreto in memoria, effettuato tramite l'invocazione del costruttore dedicato (`o_mast`).
- **Accesso alle variabili di istanza:** le regole sintattiche che permettono di leggere o manipolare i dati interni usando getter e setter.

- **Invocazione dei metodi:** la capacità dell'oggetto di richiamare le funzioni definite all'interno della propria classe di appartenenza, elaborando i parametri in ingresso e operando sul proprio stato interno.

2.4 Gestione dello Scope

La gestione dello *scope* (l'ambito di visibilità e il ciclo di vita delle variabili) rappresenta storicamente uno degli aspetti più complessi e delicati nella progettazione e implementazione di un compilatore. In molti linguaggi sperimentali o didattici, per semplificare l'architettura del parser e dell'analisi semantica, si opta per uno scope globale o per una gestione a singolo livello (solo funzioni e variabili globali).

Per *Scartellato*, al contrario, si è deciso di affrontare apertamente questa sfida ingegneristica senza ricorrere a scorciatoie. Il linguaggio implementa una vera e propria gestione gerarchica a **multi-scope**, garantendo un isolamento rigoroso delle variabili all'interno dei blocchi di codice in cui vengono dichiarate.

Questa scelta progettuale avvicina notevolmente il comportamento di *Scartellato* a quello di linguaggi di programmazione di livello *enterprise*, come Java o C++. Nel nostro linguaggio, lo scope è pervasivo e si applica a ogni costrutto strutturale:

- **Classi e Funzioni:** Ogni classe (Robba) definisce uno scope per le proprie variabili d'istanza e metodi. A loro volta, le funzioni (Mestier) e il costruttore (`o_mast`) generano un ambiente isolato per i propri parametri e le variabili locali.
- **Costrutti di controllo e iterazione:** I blocchi condizionali (come gli `if/else`) e i cicli (`aspe`, `ambress_ambress`) aprono a loro volta un nuovo sotto-scope. Una variabile dichiarata all'interno di un ciclo, ad esempio, "nasce" e "muore" esclusivamente in quel blocco, liberando la tabella dei simboli al termine dell'iterazione e prevenendo collisioni di nomi (shadowing involontario) con l'ambiente esterno.

Il cuore architetturale: Operazioni avanzate sulla Symbol Table

la risoluzione dei conflitti di **scope** sono interamente delegate a una *Symbol Table* (Tabella dei Simboli) dinamica, progettata e implementata come una struttura dati a stack. L'analisi semantica interagisce costantemente con questa struttura attraverso un set rigoroso di operazioni primitive:

- **Operazioni di Push e Pop (`enter_scope` e `exit_scope`):** Durante la visita dell'Albero Sintattico Astratto (AST), ogni volta che il compilatore incontra l'apertura di un nuovo blocco (sia esso una funzione, una classe, un condizionale o un ciclo), la Symbol Table esegue un'operazione di *push*, allocando un nuovo dizionario locale in cima allo stack. Analogamente, alla chiusura del blocco, l'operazione di *pop* distrugge l'ambiente corrente, garantendo il corretto ciclo di vita delle variabili.
- **Risoluzione gerarchica e Shadowing:** Quando il codice referencia una variabile, la Symbol Table esegue un'operazione di *lookup* che parte dall'ambito più interno

(la cima dello stack) risalendo progressivamente verso i blocchi superiori (padri). Questo meccanismo consente di distinguere con assoluta precisione se si sta accedendo a una variabile locale oppure a un'altra omonima dichiarata in un ambito esterno. Se esiste una variabile locale con lo stesso nome, essa oscura temporaneamente (*shadowing*) quella esterna nell'ambito corrente, senza corrompere i dati nei blocchi circostanti.

- **Gestione del Pending tramite Symbol Table:** Un'ulteriore criticità brillantemente superata riguarda la gestione dei riferimenti anticipati (**forward references**), come le chiamate a funzioni definite testualmente più in basso nel file rispetto al punto in cui vengono invocate. Sfruttando la Symbol Table, il compilatore memorizza le entità non ancora definite in uno stato provvisorio (*pending*), permettendo al parser di superare la prima fase di analisi senza generare errori prematuri. Una volta incontrata la definizione effettiva, la Tabella dei Simboli convalida e risolve i riferimenti pendenti, garantendo la coerenza semantica globale del programma.

3

Esempi di applicazione del linguaggio con relativa traduzione in C

Di seguito per aiutare la comprensione del linguaggio vengono riportate alcuni esempi di applicazioni per programmi noti . La struttura di questa sezione è la seguente: codice in *Scartellato* e di seguito la sua traduzione in C. La scelta del C è data dal fatto che il linguaggio è stato "transpillato" in C. Di conseguenza supporta nativamente costrutti come la ricorsione che permettono di alzare ulteriormente la complessità del linguaggio.

3.0.1 Fibonacci iterativo

Esempi di codice

```
numr mestier fibonacci_iterativo ) guagliuni : numr n ( {
    mettimmcà ) n <= 1 ( {
        ccàsta n !
    }

    numr primo = 0 !
    numr secondo = 1 !
    numr prossimo = 0 !
    numr i = 2 !

    aspe ) i <= n ( {
        prossimo = primo + secondo !
        primo = secondo !
        secondo = prossimo !
        i++ !
    }

    ccàsta secondo !
}

Uè ) ( {
    numr pos = 10 !
    numr res = jamm_ja : fibonacci_iterativo ) guagliuni : pos ( !
    arape_a_vocca ) ??Fibonacci iterativo in posizione 10: ?? - res ( !
}
```

```
#include <stdio.h>
```

```
int fibonacci_iterativo(int n) {
    if (n <= 1) {
        return n;
    }
```

```
    int primo = 0;
    int secondo = 1;
    int prossimo = 0;
    int i = 2;
```

```
    while (i <= n) {
```

```

        prossimo = primo + secondo;
        primo = secondo;
        secondo = prossimo;
        i++;
    }

    return secondo;
}

int main() {
    int pos = 10;
    int res = fibonacci_iterativo(pos);

    printf("Fibonacci_iterativo_in_posizione_10:_%d\n", res);
    return 0;
}

```

3.0.2 Fibonacci ricorsivo

Esempi di codice

```

numr mestier fibonacci ) guagliuni : numr n ( {
    // Caso base: se n <= 1 ritorna n
    mettimmcà ) n <= 1 ( {
        ccàsta n !
    }

    // In Scartellato calcoliamo gli argomenti in variabili temporanee
    numr n1 = n - 1 !
    numr n2 = n - 2 !

    numr f1 = jamm_ja : fibonacci ) guagliuni : n1 ( !
    numr f2 = jamm_ja : fibonacci ) guagliuni : n2 ( !

    numr somma = f1 + f2 !
    ccàsta somma !
}

Uè ) ( {
    numr limite = 7 !
    numr risultato = jamm_ja : fibonacci ) guagliuni : limite ( !
    arape_a_vocca ) ??Il numero di Fibonacci e: ?? - risultato ( !
}

```

```
#include <stdio.h>
```

```
int fibonacci(int n) {  
    // Caso base: se n <= 1 ritorna n  
    if (n <= 1) {  
        return n;  
    }  
  
    // Calcoliamo gli argomenti e le chiamate in variabili temporanee  
    int n1 = n - 1;  
    int n2 = n - 2;  
  
    int f1 = fibonacci(n1);  
    int f2 = fibonacci(n2);  
  
    int somma = f1 + f2;  
    return somma;  
}  
  
int main() {  
    int limite = 7;  
    int risultato = fibonacci(limite);  
  
    printf("Il_numero_di_Fibonacci_e:_%d\n", risultato);  
    return 0;  
}
```

3.0.3 Bubble Sort

Esempi di codice

```

vacant mestier bubbleSort ) guagliuni : numr ][ arr, numr dim ( {
    numr i = 0 !
    aspe ) i < dim ( {
        numr j = 0 !
        aspe ) j < dim - i - 1 ( {
            mettimmcà ) arr[j] > arr[j - 1] ( {
                // Swap tra elementi dell'array
                arr[j] <-> arr[j - 1] !
            }
            j++ !
        }
        i++ !
    }
}

Uè ) ( {
    numr ][ numeri = 0 !
    numeri -= 45 !
    numeri -= 12 !
    numeri -= 89 !
    numeri -= 3 !

    jamm_ja : bubbleSort ) guagliuni : numeri, 4 ( !
    arape_a_vocca ) ??Array ordinato: ?? - numeri]0[( !
}

```

```
#include <stdio.h>
```

```

void bubbleSort(int arr[], int dim) {
    int i = 0;
    while (i < dim) {
        int j = 0;
        while (j < dim - i - 1) {
            if (arr[j] > arr[j - 1]) {
                // Swap tra elementi dell'array
                int temp = arr[j];
                arr[j] = arr[j - 1];
                arr[j - 1] = temp;
            }
            j++;
        }
        i++;
    }
}

```

```
        }
        j++;
    }
    i++;
}

}

int main() {
    int numeri[4];
    numeri[0] = 45;
    numeri[1] = 12;
    numeri[2] = 89;
    numeri[3] = 3;

    bubbleSort(numeri, 4);

    printf("Array_ordinato:_%d\n", numeri[0]);
    return 0;
}
```

4

Utilizzo dell'IA come Strumento di Supporto nello Sviluppo

4.1 Filosofia d'Uso: L'IA come Tool e non come Sostituto dell'Uomo

Nel corso della realizzazione di *Scartellato*, l'Intelligenza Artificiale è stata impiegata seguendo il principio che l'IA deve servire da strumento di supporto (tooling) per velocizzare il lavoro e come fonte di conoscenza, senza mai sostituire l'approccio critico e il cervello umano.

La progettazione dell'architettura, la definizione delle regole sintattiche del linguaggio e le scelte di design restano interamente sotto la responsabilità del developer. L'IA è intervenuta come un assistente avanzato per accelerare la scrittura di codice, suggerire pattern architetturali standard e automatizzare compiti operativi complessi che avrebbero richiesto un notevole dispendio di tempo.

4.2 Supporto nell'Implementazione del Pattern Visitor

Una delle fasi maggiormente facilitate dall'impiego dell'IA è stata l'implementazione del **Pattern Visitor**.

L'IA ci ha fornito un supporto concreto in:

- **Analisi Semantica:** Dato che gestire multi-scope classi e tipi dinamici è stata una impresa a dir poco ardua, essa è stata in grado di aiutarci a strutturare le classi del Visitor deputate a percorrere i nodi dell'albero sintattico, verificando la correttezza dello scope, la presenza delle funzioni (*mestier*) e delle classi (*robba*).
- **Type Checker:** Durante le varie fasi di parsing e visita dell'AST, l'IA ha aiutato a individuare casi limite (*edge cases*) relativi alla gestione degli scope e alla risoluzione

delle espressioni matematiche con operatori invertiti.

- **Transpiler** la traduzione in C può risultare complessa per i tipi generici e classi per cui ci è stata di grande mano per gestire le ripetitive e verbose traduzioni del C. **compiti di natura ripetitiva e codice boilerplate**

4.3 Il Transpiler: Traduzione dal Codice Scartellato al Linguaggio C e Gestione dei Compiti Ripetitivi

Per l'esecuzione dei programmi scritti in *Scartellato*, si è scelto di progettare un **Transpiler** (o traduttore da sorgente a sorgente). Come linguaggio target di destinazione è stato scelto il linguaggio C, per via delle sue elevate prestazioni, della sua trasparenza nella gestione della memoria e della sua universale portabilità.

4.4 Supporto nel Debugging e Correzione della Traduzione dei Tipi Generici

Oltre all'assistenza nella stesura iniziale del codice, l'Intelligenza Artificiale si è rivelata uno strumento indispensabile nelle fasi di **debugging e risoluzione degli errori** emersi durante le analisi e la traspilazione.

In particolare, l'IA ha svolto un ruolo chiave nei seguenti ambiti di correzione:

- **Correzione degli Errori nell'Analisi Semantica e nel Type Checker:**
- **Gestione degli Array di Tipi Generici** (`burdell [] []`): La combinazione della struttura dati array dinamico con il tipo generico ha generato diversi errori in fase di allocazione e ridimensionamento della memoria in C. L'IA ha fornito un supporto determinante nella correzione della logica di generazione C per le operazioni di *append* (`-=`) e *rimozione* (`+=`), garantendo che le funzioni `malloc`, `realloc` e la gestione dei puntatori venissero tradotte in modo corretto e robusto.

4.5 Automazione della Compilazione tramite GCC

Una volta completata la generazione del codice sorgente C da parte del Transpiler, è necessario trasformarlo in un file eseguibile binario.

Anche in questa fase l'IA è stata utilizzata come strumento di automazione:

- Ci ha supportato nell'implementazione di uno script/routine che rileva e individua automaticamente il compilatore **GCC** presente nell'ambiente di sistema dell'utente.
- Avvia autonomamente il processo di compilazione passando a GCC il codice C generato dal Transpiler.
- Gestisce la cattura di eventuali errori di compilazione di basso livello, fornendo all'utente il binario pronto per l'esecuzione.

Attraverso questa pipeline guidata, lo sviluppatore può scrivere in *Scartellato* ed eseguire direttamente il proprio programma, sfruttando l'IA come un solido ponte tra l'ideazione di alto livello e l'esecuzione di basso livello.

4.6 Apprendimento e Supporto nel Testing del Compilatore

Un aspetto di inestimabile valore aggiunto è arrivato dall'affiancamento dell'IA nella fase di **testing del compilatore**.

Poiché la stesura della gran parte dei test case è stata realizzata con il supporto attivo dell'IA, è stato interessante apprendere concretamente come si progetta e si struttura una suite di test per un linguaggio di programmazione, coprendo casi limite sintattici, semantici e prestazionali che garantiscono la robustezza dell'intero sistema.

5

Testing e Risultati finali

Per la fase di verifica e validazione del sistema è stata adottata la metodologia di testing basata sul *Category-Partition*, progettata per testare in modo sistematico ed esaustivo tutte le funzionalità, i costrutti sintattici e le particolarità semantiche del linguaggio.

L'intera campagna di testing si compone di **27 programmi di prova**, suddivisi strategicamente in **7 macro-categorie** per garantire una copertura completa di ogni modulo del compilatore:

- **Test costruito condizionale** : 5 programmi
- **Test operatori matematici**: 4 programmi
- **Test variabili e tipi dinamici (burdell)**: 4 programmi
- **Test programmazione a oggetti (robba)**: 5 programmi
- **Test operatori e costrutti avanzati burdell**: 5 programmi
- **Test gestione funzioni e scope**: 3 programmi
- **Test operazioni di scambio (swap)**: 1 programma

Di seguito sono allegate le metriche:

Totale complessivo: 27 test case validati con successo.

```
==== Riepilogo: 26 test totali ====  
OK: 25 | FALLITI: 0 | DA VERIFICARE: 0 | MANUALI (scanf): 1  
Validity rate (pass/totale): 96.2%
```

```
===== COVERAGE GRAMMATICA =====  
Regole totali osservabili: 33  
Regole coperte dai test: 27 (81.8%)
```